

# Informe Consolidado de Hardening y Seguridad (Proyecto Unidad 3)

## Universidad de Talca - Administración de Redes y Sistemas Computacionales

**Objetivo:** Este informe documenta el cumplimiento de los requisitos de seguridad y hardening aplicados sobre la arquitectura de microservicios desarrollada en el Proyecto 2, enfocándose en las mejores prácticas de seguridad empresarial.

### 1. HTTPS/TLS - Comunicación Segura

#### 1.1. Implementación

Se ha implementado HTTPS/TLS en el API Gateway (Nginx) para asegurar toda la comunicación de entrada y salida de la arquitectura.

- **Generación de Certificados:** Se utiliza el script `scripts/security/generate-dev-cert.sh` para generar certificados autofirmados (`cert.pem` y `key.pem`) con SAN para `dev.local` y `127.0.0.1`, válidos para el entorno de desarrollo.
- **Configuración del API Gateway (`services/api-gateway/nginx.conf`):**
  - **Redirección:** El puerto 8080 (HTTP) está configurado exclusivamente para realizar una redirección 301 permanente al puerto 8443 (HTTPS).
  - **Protocolos y Cifrados:** Se restringe el uso a protocolos fuertes (TLSv1.2 y TLSv1.3) con una lista de cifrados seguros.

#### 1.2. Headers de Seguridad

El API Gateway se reforzó con los siguientes *headers* de seguridad para mitigar ataques comunes del lado del cliente:

| Header                                        | Valor                                            | Función                                                                   |
|-----------------------------------------------|--------------------------------------------------|---------------------------------------------------------------------------|
| <code>Strict-Transport-Security</code> (HSTS) | <code>max-age=31536000; includeSubDomains</code> | Obliga a los clientes a usar HTTPS en el futuro.                          |
| <code>X-Frame-Options</code>                  | <code>SAMEORIGIN</code>                          | Previene ataques de Clickjacking.                                         |
| <code>X-Content-Type-Options</code>           | <code>nosniff</code>                             | Previene ataques de Mime-Type Sniffing.                                   |
| <code>Referrer-Policy</code>                  | <code>no-referrer-when-downgrade</code>          | Controla cuándo la información del referrer se incluye en las peticiones. |
| <code>Content-Security-Policy</code> (CSP)    | <code>upgrade-insecure-requests;</code>          | Indica al navegador actualizar peticiones HTTP a HTTPS.                   |

#### 1.3. Evidencias y Verificación

- **Verificación de Redirección (301):**

```
curl -k -I http://localhost:8080
# ...
# HTTP/1.1 301 Moved Permanently
# Location: https://localhost:8443
# ...
```

- **Verificación de Headers (Extracto):**

```
curl -k -I https://localhost:8443 | findstr /R "Strict-Transport-
Security\|Content-Security-Policy\|X-Frame-Options"
# Muestra: Strict-Transport-Security: max-age=31536000; includeSubDomains
# Muestra: X-Frame-Options: SAMEORIGIN
# Muestra: Content-Security-Policy: default-src 'self'; ... upgrade-
insecure-requests;
```

## 2. Hardening de Contenedores Docker

Se aplicaron principios de mínimo privilegio a los contenedores de la arquitectura.

### 2.1. Requisitos de Hardening

| Requisito                   | Aplicación                                                                                             | Archivo/Estado     |
|-----------------------------|--------------------------------------------------------------------------------------------------------|--------------------|
| Usuario no privilegiado     | Implementado en servicios Node.js, DB-proxy, y la pila de monitoreo (Grafana/Loki).                    | docker-compose.yml |
| Imágenes base específicas   | Implementado. Servicios Node.js usan imágenes con tag explícito y estable, como node:18.18.2-alpine.   | Dockerfiles        |
| Multi-stage builds          | Implementado en servicios Node.js para reducir la superficie de ataque y el tamaño de la imagen final. | Dockerfiles        |
| security_opt/cap_drop       | security_opt: no-new-privileges y cap_drop: [ALL] aplicadas a backends y servicios de monitoreo.       | docker-compose.yml |
| Escaneo de Vulnerabilidades | la generación y adjunto del reporte de Trivy/Docker Scout/Grype.                                       |                    |

### 2.2. Justificación para Excepción (Nginx)

Se removió temporalmente no-new-privileges/cap\_drop y se permite la ejecución como root (user: "0:0") en los contenedores Nginx de frontends y api-gateway.

- **Motivo:** La imagen base nginx:alpine realiza un chown sobre /var/cache/nginx/\* en su entrypoint. Al utilizar un usuario no root con la directiva no-new-privileges activada, este proceso falla con el

error `chown(...) failed (1: Operation not permitted)`, provocando un bucle de reinicio del contenedor.

- **Mitigación Futura:** Se planea montar `tmpfs` en `/var/cache/nginx` con permisos adecuados o construir una imagen derivada donde la estructura de caché se inicialice correctamente por un usuario no root.

### 3. Hardening de Bases de Datos (PostgreSQL)

#### 3.1. Medidas de Seguridad Aplicadas

- **Aislamiento de Red:** El servicio `postgres-master` no tiene puertos expuestos al host y solo es accesible a través de la red interna `database-network`. El acceso está limitado a los contenedores de aplicación (`db-proxy`, `backends`) por red interna.
- **Reglas `pg_hba.conf`:** Las reglas de acceso están limitadas a la CIDR interna de Docker (`172.21.0.0/16`). Se exige el método de autenticación `scram-sha-256` para roles de aplicación y réplica, bloqueando métodos obsoletos como `md5`. El superusuario solo puede acceder localmente (`local`).
- **Usuario de Aplicación con Privilegios Mínimos:**
  - Se utiliza un rol específico (`APP_DB_USER`) con una contraseña de **≥ 12 caracteres**.
  - El rol **`solo`** tiene permisos `SELECT`, `INSERT`, `UPDATE`, `DELETE` en el esquema `public`, sin permisos administrativos (`SUPERUSER`, `CREATEDB`, `CREATEROLE`, `REPLICATION`).
- **Logging y Auditoría:** `log_connections`, `log_disconnections`, y `log_duration` están habilitados para el seguimiento de la actividad. Los logs se envían a `stderr` para ser recolectados por el sistema centralizado (Loki).

#### 3.2. Evidencias de Aislamiento y No Exposición

| Verificación                 | Comando (o evidencia)                                                                                                                 | Resultado y Comentario                                                                                                                                                                                                |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Puerto no expuesto           | <code>docker compose ps</code>                                                                                                        | La columna <i>Ports</i> para <code>postgres-master</code> muestra solo el puerto interno <code>5432/tcp</code> sin <i>binding</i> al host (host local no puede conectar).                                             |
| Acceso Denegado (Desde Host) | <code>psql -h host.docker.internal -U user -d municipalidad_db -c "select 1;"</code>                                                  | <b>Salida:</b> <code>psql: error: connection to server... FATAL: la autenticacion password fallo.</code> (El acceso se rechaza, incluso si se usara el puerto interno, porque no está expuesto y exige credenciales). |
| Conexión Interna Exitosa     | <code>docker exec -e PGPASSWORD=user123_adminX -it postgres-master psql -h db-proxy -U user -d municipalidad_db -c "select 1;"</code> | <b>Salida:</b> <code>(1 row)</code> - Demuestra que el acceso funciona dentro de la red interna de Docker con credenciales y autenticación correctas.                                                                 |

### 4. Gestión de Secretos

## 4.1. Implementación y Documentación

- **Almacenamiento:** Se utiliza la gestión de credenciales a través de **Variables de Entorno** cargadas desde el archivo `.env`.
  - **Control de Versiones:** El archivo `.env` está incluido en el `.gitignore`. Se mantiene un archivo `.env.example` con *placeholders* en el repositorio para documentación.
  - **Rotación de Secretos:** Se ha creado el script `scripts/security/rotate-secrets.sh` para generar y actualizar credenciales fuertes para la DB, JWT, Redis, etc., facilitando la rotación periódica.
- 

## 5. WAF y Seguridad de Red

### 5.1. Implementación en el API Gateway (Nginx)

- **Rate Limiting:** Configurado en `services/api-gateway/nginx.conf` (`limit_req`) con un límite de **10 peticiones por segundo** y un *burst* de 20 por IP (`limit_req_zone perip`), aplicado a los *endpoints* `/api/...`
- **Bloqueo de User-Agents:** El Gateway bloquea (403 Forbidden) *User-Agents* maliciosos conocidos, como "sqlmap".
- **Métodos HTTP:** Se restringen los métodos permitidos (GET, POST, PUT, DELETE, HEAD, OPTIONS); cualquier otro método devuelve 405.
- **Segmentación de Redes:** Se utiliza la segmentación en `frontend-network`, `backend-network`, y `database-network` en `docker-compose.yml` para un aislamiento efectivo del tráfico.

### 5.2. Evidencias de Bloqueo (WAF)

- **Bloqueo por User-Agent Malicioso (403 Forbidden):**

```
curl.exe -k -I -A "sqlmap" https://localhost:8443/api/auth/  
# Respuesta:  
# HTTP/1.1 403 Forbidden  
# ... (Headers de seguridad)
```

La entrada en los logs del gateway registra el evento como `blocked_ua=1`, confirmando la activación de la regla de User-Agent.

- **Rate Limiting (429 Too Many Requests):** Se confirmó que al lanzar una ráfaga alta (ej. 200 peticiones en bucle) sobre un *endpoint* protegido (`/api/auth/`), se reciben respuestas `429 Too Many Requests` en la consola.

```
# Fragmento de log tras ráfaga (confirmando activación del límite):  
# ... [error] 13#13: *6 limiting requests, excess: 20.000 by zone "perip",  
client: 172.21.0.1, server: 0.0.0.0:443 ...
```

---

## 6. Análisis de Vulnerabilidades

- Estado:** El análisis de vulnerabilidades y la documentación de remediación están de completarse en su totalidad.
- **Escaneo de imágenes Docker (Trivy/Grype):** de ejecución y generación del reporte.
  - **Escaneo de dependencias (npm audit, pip safety):** de ejecución.
  - **Análisis web básico (Nikto):** de ejecución.
  - **Documento de Remediación:** de redactar (debe listar vulnerabilidades **HIGH/CRITICAL** y su corrección o justificación).
  - **Script Automatizado:** de crear el script `scripts/security/scan-vulnerabilities.sh` para integrar las herramientas de escaneo.

## 7. Logging y Auditoría de Seguridad

### 7.1. Sistema Centralizado

- **Sistema:** Se utiliza la pila **Loki + Promtail + Grafana** para la recolección, indexación y visualización de logs de seguridad.
- **Configuración:** El archivo `promtail-config.yml` clasifica y enriquece los logs:
  - **job=security-gateway:** Logs del API Gateway (Nginx). Se utilizan *pipelines* para parsear el formato de acceso y mantener eventos clave como **401/403** (accesos denegados, WAF).
  - **job=security-app:** Logs de los microservicios (auth/config/etc.). Se parsea si la aplicación loguea en formato JSON básico (niveles, *status*, ruta, usuario).
- **Eventos Registrados:** Intentos de autenticación, accesos denegados (401, 403), uso de *endpoints* sensibles, y conexiones a la base de datos (vía logs de Postgres: `log_connections`, `log_disconnections`).

### 7.2. Herramientas de Consulta

- **Script de consulta:** `scripts/security/query-logs.sh` para facilitar la interacción con Loki.
  - *Ejemplo de uso:* `scripts/security/query-logs.sh '{job="security-gateway"} |= "403"'` (Filtra bloqueos del WAF).

## 8. Políticas y Documentación de Seguridad

Se han creado los documentos obligatorios del proyecto.

| Documento Obligatorio                                                 | Estado | Observación                                                                        |
|-----------------------------------------------------------------------|--------|------------------------------------------------------------------------------------|
| 1. Política de Seguridad ( <code>politica-seguridad.md</code> )       |        | Debe completarse con políticas formales de contraseñas, retención de logs, etc.    |
| 2. Matriz de Riesgos ( <code>matriz-riesgos.md</code> )               |        | Falta la identificación y mitigación de los riesgos requeridos.                    |
| 3. Checklist OWASP Top 10 ( <code>owasp-top10.md</code> )             |        | Falta completar la matriz de mitigación para cada riesgo del OWASP 2021.           |
| 4. Plan de Respuesta a Incidentes ( <code>plan-incidentes.md</code> ) |        | Falta la redacción de los procedimientos de detección, contención, y recuperación. |

| Documento Obligatorio                                                      | Estado | Observación                                                                  |
|----------------------------------------------------------------------------|--------|------------------------------------------------------------------------------|
| 5. Cumplimiento Normativo<br>( <a href="#">cumplimiento-normativo.md</a> ) |        | Falta la documentación de cumplimiento para las Leyes N° 19.628 y N° 21.459. |

**Documentación Técnica Adicional Incluida:**

- [README.md](#) (Actualizado con flujo de trabajo, manejo de secretos y arranque).
- [docs/hardening-contenedores.md](#) (Justificación de excepciones).
- [docs/hardening-database.md](#) (Detalle de medidas en PostgreSQL).
- [docs/logging-seguridad.md](#) (Detalle de la configuración de Loki/Promtail).